CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Trivia & Quiz Night Manager

Project Design Document

Group Name / No.:	2
Members:	*Justine Cuizon*
	Clarence Philip Elegino
	Joshua Malaya
	Gebrielle Mirasol
	John Daevin Simbajon
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	*V0.1*

Table of Contents

(Right-click anywhere in this box and choose "Update Field" -- or press F9 -- after finishing the document, to generate the real table of contents.)

Right-click and select "Update Field" to generate the Table of Contents.

Revision History

Update this table each week as your team adds or revises sections of this document.

Version	Date	Author	Changes
0.1	10/24/26	Group 2	Initial draft — Week 8 sections
0.2			Week 9 sections added
0.3			Week 10 sections added
1.0			Final compilation and polish — Week 11

Formatting Guidelines

Follow these formatting rules consistently throughout the document so the final compiled version reads as one polished deliverable, not four stitched-together drafts.

Element	Rule
Body font	Calibri, 11pt (the default font of this template — do not mix fonts)
Headings	Use Word's built-in Heading 1/2/3 styles (already applied to every section title below) — this is required for the Table of Contents field to work
Margins	1 inch on all sides
Line spacing	1.15–1.5 line spacing for body paragraphs; singlespacing inside tables
Table captions	Label every table "Table N: short description" directly ABOVE the table, numbered sequentially from Table 1 through the end of the document
Figure/chart captions	Label every chart/screenshot "Figure N: short description" directly BELOW the figure, numbered sequentially from Figure 1
Code snippets	Use a monospace font (Consolas or Courier New), 9–10pt, and keep each snippet under ~20 lines — link to the full file in your repository instead of pasting it all
Page numbers	Already automatic — every page footer includes a live page number field
Alignment	Body text left-aligned/justified; headings left-aligned; do not center body paragraphs

1. Introduction & Project Overview

Describe what your app does, who it's for, and which theme (or custom idea) your group chose. Summarize the problem being solved and the app's key value proposition in 2-4 sentences. This section is first drafted in Week 8 and should need only light editing later.

- Trivia & Quiz Game Night Manager is a Java desktop application that helps friends, classmates, and community groups organize and run trivia nights. Players enter their names, choose a category, and answer questions fetched live from the Open Trivia Database, with an “undo last answer” option for mistakes. Every player, game session, and score is saved in a local SQLite database, and past score history can be imported from CSV files. The app then analyzes the saved results and shows average score per category, most-missed questions, and score trends over time as labeled bar and line charts. Theme: Custom idea (an entertainment and social-gaming tool for trivia nights).

  Problem solved: Quiz hosts usually keep score on paper or in scattered spreadsheets, so results are easy to lose and hard to compare from one night to the next.

  Key value proposition: One app that fetches questions, scores each player, stores the history, and turns it into clear charts, so groups can see who is improving and which categories are hardest.

2. Preliminary ER Sketch & Team Roles

Insert the preliminary entity-relationship sketch from your Week 8 proposal — entities, key attributes, and at least one relationship — and list every team member's name and primary role. This is a rough draft; it will be superseded by the finalized ERD in Section 4.

Entities and key attributes

- Category(id, name, parent_id FK → Category.id)
- Player(id, name)
- Question(id, text, correct_answer, incorrect_answers, difficulty, category_id FK)
- GameSession(id, played_at)
- Score(id, session_id FK, player_id FK, category_id FK, question_id FK (nullable), is_correct, points)

Relationships

Category --1:M--> Category (parent_id: a category has many subcategories)

Category --1:M--> Question (Question.category_id)

Category --1:M--> Score (Score.category_id)

Question --1:M--> Score (Score.question_id, optional for imported CSV rows)

Player(id, name) --1:M--> Score(id, session_id FK, player_id FK, category_id FK, question_id FK, is_correct, points) <--M:1-- GameSession(id, played_at)

Score resolves the many-to-many relationship between Player and GameSession: a player joins many sessions, and a session has many players.

Table 1: Team members and primary roles

Member	**Primary Role**
John Daevin S. Simbajon	*Team Leader: project structure, schema, GitHub repo and branch rules, JavaFX MainApp shell, pull request reviews*
Gebrielle P. Mirasol	*Model: abstract Question class, ScienceQuestion, HistoryQuestion, EntertainmentQuestion, plus Player and Category*
Justine B. Cuizon	*Database and API: Database class, DAO classes (PreparedStatement), Open Trivia Database client and JSON parsing*
Clarence Philip A. Elegino	*Game Logic: GameEngine (Queue for questions, Stack for undo), recursive category-tree method, JUnit tests*
Joshua Malaya	*Analytics and Testing: data mining methods, AnalyticsService tests, charts*

3. System Architecture & Object-Oriented Design

Present your class hierarchy (a diagram or a clear textual description) showing the abstract class/interface and its concrete subclasses, and justify how each class follows the Single Responsibility Principle. Explain the design rationale in a short paragraph.

4. Data Model & Database Design

Include the finalized ER diagram (3+ entities, at least one relationship with correct cardinality) and the actual SQLite schema (CREATE TABLE statements or an equivalent schema table). This section supersedes the preliminary sketch from Section 2.

5. API Integration Design

Name the public API your app consumes, describe the endpoint(s) called, the shape of the JSON response, and how that data is parsed and used meaningfully in the app.

6. Concurrency Design

Identify the long-running operation(s) that run on a background thread, explain the threading approach used, and describe how the UI stays responsive during that operation.

7. Data Mining & Visualization Design

Describe the aggregation or frequency-analysis step performed on your app's data and the chart(s) used to visualize the result, including why that chart type was chosen.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

(Add any additional references, links to full API documentation, licensing notes, or supplementary material here.)